ПОЧЕМУ ИМЕННО ЭТИ ПРИЗНАКИ

Развернутое обоснование отбора 27 findings для защиты ВКР

Тема: объяснимый статический анализ x64 PE exception/unwind-механизма

Главный вопрос комиссии	Краткий ответ
Почему именно 27?	Потому что они покрывают все критические слои x64 PE exception/unwind: PE layout, exception directory, RUNTIME_FUNCTION table, UNWIND_INFO, handler layer, placement и корреляции.
Почему не больше?	Потому что дополнительные признаки часто уходят в общий антивирусный PE-анализ или дублируют уже существующие сигналы.
Почему не меньше?	Потому что исчезли бы слепые зоны: без PE layout нет координат, без .pdata нет карты функций, без UNWIND_INFO нет раскрутки стека, без placement/correlation нет анализа подозрительной компоновки.
На основании чего отбирались?	На основании структуры x64 Windows exception/unwind, проверяемых инвариантов PE, практической полезности для reverse engineering, ручной верифицируемости и объяснимой evidence matrix.

Дата подготовки: 14.07.2026 20:05

1. Короткая защитная позиция

Если комиссия задает вопрос 'почему именно эти признаки', отвечать нужно не с перечисления кодов, а с границы предметной области. Проект не является универсальным антивирусом и не пытается оценивать весь PE-файл по всем возможным malware-индикаторам. Его предмет - x64 Windows exception/unwind-механизм: таблица .pdata, записи RUNTIME_FUNCTION, структуры UNWIND_INFO, flags обработчиков исключений, HandlerRVA и размещение этих структур в PE-образе.

Поэтому признаки выбраны не произвольно, а как инженерное покрытие полного пути анализа. Сначала нужно проверить, что PE-координаты имеют смысл. Потом - что exception directory ведет к читаемой function table. Потом - что каждая функция имеет корректный диапазон и ссылку на UNWIND_INFO. Потом - что UNWIND_INFO можно разобрать и что внутри нет логических аномалий. Затем - что handler layer выглядит правдоподобно. И наконец - что подозрительные сигналы не одиночные, а коррелируют между слоями.

27 - это не магическое число. Это результат декомпозиции механизма на проверяемые слои и исключения лишних признаков, которые не относятся напрямую к теме ВКР. Важно говорить именно так: не 'я придумал 27 признаков', а 'я выделил слои механизма и для каждого слоя взял минимально достаточные проверяемые инварианты и практические эвристики'.

2. Что именно является объектом анализа

- PE layout - нужен, чтобы понять, существуют ли корректные file offset и RVA-пространство. Если section table противоречива, анализ .pdata/.xdata становится ненадежным.

- Exception Directory - это data directory, через который x64 PE сообщает расположение таблицы RUNTIME_FUNCTION.

- RUNTIME_FUNCTION - карта диапазонов функций. Она связывает BeginAddress/EndAddress с UnwindInfoAddress.

- UNWIND_INFO - описание пролога функции, stack frame, сохраненных регистров, chained info и optional handler.

- Handler layer - слой EHANDLER/UHANDLER, где появляется HandlerRVA и language-specific data.

- Placement - проверка, где лежит metadata: в data-секции, executable-секции, writable-секции или необычной области.

- Correlation - проверка сочетаний: один признак может быть спорным, а комбинация признаков по разным слоям уже намного убедительнее.

Именно эти объекты образуют техническую цепочку: PE image -> exception directory -> RUNTIME_FUNCTION -> UNWIND_INFO -> handler -> итоговая интерпретация. Признаки проекта соответствуют этой цепочке.

3. Критерии включения признака в матрицу

Критерий	Что означает	Почему это важно для ВКР
Предметная релевантность	Признак должен относиться к x64 exception/unwind, а не к общему malware analysis.	Иначе проект расплывется в антивирусную область и потеряет научную точность.
Проверяемый инвариант	Признак должен проверять понятное правило: границы, порядок, пересечение, читаемость, section permissions.	Комиссия может спросить 'почему это ошибка', и на это должен быть технический ответ.
Статическая измеримость	Признак должен извлекаться из PE без обязательного запуска файла.	Главная часть проекта - статический анализ, поэтому критерии должны работать воспроизводимо.
Ручная проверяемость	Аналитик должен суметь подтвердить finding в PE viewer, Ghidra, hex view или debugger.	Это делает проект объяснимым, а не черным ящиком.
Минимальное дублирование	Новый признак не должен просто повторять уже существующий под другим названием.	Иначе 27 превратились бы в раздутый список.
Практическая ценность	Finding должен помогать reverse engineer понять, где именно сломана или необычна unwind/EH-логика.	Отчет должен вести к действию, а не просто к красивой статистике.
Управляемый false positive	Если у признака есть benign-объяснения, должны быть mitigations, thresholds или низкий вес.	Так проект не обвиняет файл по одному слабому сигналу.
Вклад в verdict	Признак должен участвовать в score, confidence, family coverage или correlation.	Иначе он лучше подходит для informational note, а не finding.
Тестируемость	Для признака должен быть возможен тестовый или синтетический бинарный пример.	Иначе сложно доказать, что признак реально работает.

Эти критерии важны потому, что они отвечают на главный упрек: 'разработчик просто придумал набор признаков'. Да, разработчик формирует модель, но признаки не выбираются произвольно. Они проходят фильтр предметной релевантности, технической проверяемости и практической пользы.

4. Критерии исключения: почему не добавлены другие признаки

Чтобы обосновать 27 признаков, нужно объяснять не только включение, но и исключение. Хороший проект отличается не тем, что в нем максимум признаков, а тем, что признаки соответствуют цели. Ниже - почему некоторые очевидные PE-индикаторы не включены в scoring matrix.

Кандидат	Почему может показаться полезным	Почему не включен в 27 scoring findings
COFF Header	Machine, NumberOfSections, SizeOfOptionalHeader определяют базовый формат PE.	Это входная валидация. Если Machine не AMD64 или header не читается, анализ нужно остановить. Это не подозрительность unwind-механизма, а условие применимости инструмента.
Imports	Импорты могут показывать WinAPI, packer/runtime, криптографию.	Они не отвечают на вопрос, корректна ли .pdata, UNWIND_INFO и HandlerRVA. Это общий malware-analysis слой.
Entropy	Высокая entropy может намекать на упаковку.	Entropy не объясняет конкретную поломку exception/unwind. Это косвенный packer-сигнал, а не структурный unwind finding.
Strings	Строки могут давать контекст поведения программы.	Строки не связаны напрямую с virtual unwind, handler flags и section placement metadata.
Digital signature	Подпись важна для доверия к файлу.	Подпись не говорит, согласованы ли RUNTIME_FUNCTION и UNWIND_INFO. Подписанный файл тоже может быть обфусцирован или поврежден.
Timestamp	Нулевой или странный timestamp иногда подозрителен.	Слишком слабый и общий PE-признак, не относящийся к теме exception/unwind.
Overlay	Overlay часто используют упаковщики.	Это признак упаковки файла в целом, а не конкретной EH/unwind-структуры.
TLS callbacks	TLS может использоваться для раннего запуска кода.	Полезно для malware analysis, но не проверяет .pdata/.xdata и handler metadata.
Rich header	Может показывать toolchain.	Не является инвариантом unwind-механизма и плохо подходит для scoring конкретной темы.
Полное сравнение prolog с UNWIND_INFO	Очень сильная проверка корректности unwind.	Это уже уровень дизассемблера и полноценного анализа инструкций. Для версии ВКР выбран более стабильный статический слой без рискованного hand-rolled disassembler.
ML-классификация	Можно обучить модель на наборе бинарников.	Это сделало бы систему менее объяснимой. ВКР строится на reviewable evidence matrix, а не на черном ящике.

5. Почему COFF Header не отдельный finding

COFF Header отвечает за базовые параметры PE-файла: Machine, NumberOfSections, SizeOfOptionalHeader, Characteristics и другие поля. Для проекта особенно важны Machine и SizeOfOptionalHeader. Если Machine не AMD64, x64 unwind-логика проекта неприменима. Если SizeOfOptionalHeader или PE signature повреждены, нельзя корректно найти section table и data directories.

Но это не значит, что COFF Header должен быть одним из 27 suspicious findings. Его роль другая: это предварительный фильтр применимости. Он отвечает на вопрос 'можно ли вообще анализировать этот файл данным инструментом'. Findings отвечают на другой вопрос: 'что подозрительного или несогласованного найдено внутри поддерживаемого x64 PE exception/unwind-механизма'.

Правильная защитная формулировка: COFF/Optional Header проверяются в парсере PE как precondition. Ошибки этого слоя являются parse errors или unsupported format diagnostics, а не scoring findings. Включать их в score было бы методологически неверно, потому что неподдерживаемая архитектура не означает подозрительность, она означает невозможность корректного применения модели.

6. Послойное обоснование набора из 27

Слой	Признаки	Почему этот слой нужен	Что будет, если убрать
PE layout	1-6	Проверяет, что file offsets, RVA, SizeOfImage, section table и exception directory согласованы.	Анализатор может строить выводы на неправильной координатной системе.
Function table	7-13	Проверяет карту функций: диапазоны, порядок, пересечения, ссылки на UNWIND_INFO.	Virtual unwind не сможет надежно сопоставить RIP с функцией.
UNWIND_INFO	14-15	Проверяет читаемость и внутреннюю логику unwind metadata.	Будет непонятно, можно ли выйти из кадра и доверять prolog/handler flags.
Reuse/dominance	16-18	Ищет статистические паттерны централизации metadata и обработчиков.	Проект пропустит dispatcher-style схемы, где формат формально валиден.
Placement	19-23	Проверяет права и секции, где лежат metadata и handler targets.	Проект пропустит случаи, когда данные лежат в коде, writable memory или handler не является кодом.
Correlation	24-27	Усиливает вывод, когда подозрительные сигналы встречаются вместе.	Отчет останется плоским списком и не сможет показать связанные evidence patterns.

6.1. Почему первые 6 относятся к PE layout

Первые признаки проверяют не malware behavior, а пригодность координатной системы PE. Это похоже на проверку карты перед маршрутом. Если raw-диапазоны выходят за файл, virtual-диапазоны выходят за SizeOfImage, секции пересекаются или exception directory не читается, то дальнейшие выводы о .pdata и UNWIND_INFO могут быть ошибочными.

Эти признаки выбраны потому, что x64 unwind полностью основан на адресации через RVA. RUNTIME_FUNCTION хранит BeginAddress, EndAddress и UnwindInfoAddress как RVA. HandlerRVA тоже RVA. Поэтому любой конфликт в section table или SizeOfImage напрямую влияет на весь анализ.

6.2. Почему 7 признаков относятся к function table

RUNTIME_FUNCTION - центральная карта функций. Она должна описывать непустые диапазоны кода, находиться внутри образа, быть упорядоченной, не пересекаться и ссылаться на существующий UNWIND_INFO. Эти свойства не декоративные. Они нужны для поиска записи по RIP и для корректного virtual unwind.

Если оставить только один признак 'таблица повреждена', отчет станет бесполезным для аналитика. Ему нужно знать, что именно не так: диапазон обратный, адрес вне SizeOfImage, UnwindInfoRVA битый, порядок нарушен, диапазоны пересекаются или begin дублируется. Поэтому function table разбита на несколько конкретных findings.

6.3. Почему UNWIND_INFO представлен двумя основными признаками

UNWIND_INFO можно проверять очень детально: version, flags, count codes, opcodes, operands, chain info, handler tail, alignment, code offsets. Но если сделать отдельный finding на каждую микродеталь, матрица станет шумной и неудобной для защиты. Поэтому проект использует два уровня: unwind_parse_fail - структура не читается безопасно; unwind_anomalies - структура читается, но содержит логические аномалии.

Такой выбор сохраняет глубину анализа внутри парсера, но не раздувает внешнюю evidence matrix. Внутри отчета все равно можно видеть детали, а score получает понятные укрупненные сигналы.

6.4. Почему нужны reuse и handler dominance

Не все подозрительные случаи выглядят как битая структура. Обфускатор или защитный runtime может сделать формально корректные RUNTIME_FUNCTION и UNWIND_INFO, но использовать их необычно: слишком много функций ссылается на один UNWIND_INFO, или почти все handler-записи ведут к одному handler target. Поэтому нужны статистические pattern-признаки.

При этом проект осторожен: moderate reuse работает только при n > 200 и при наличии другого статического сигнала, strong reuse требует больше 30%, handler dominance требует handler_count > 50 и ratio > 80%. Это показывает, что признаки отбирались не для максимального количества срабатываний, а для управляемой

интерпретации.

6.5. Почему placement выделен отдельно

Даже если все поля формально читаются, важно где лежит metadata. UNWIND_INFO обычно является данными и ожидается в .xdata/.rdata/.pdata или похожих секциях. Handler target должен быть исполняемым кодом. Если metadata лежит в executable-секции, writable-секции или handler указывает в non-executable область, это отдельная важная информация для reverse engineer.

Placement-признаки нужны потому, что они ловят не синтаксическую битость, а подозрительную компоновку. Это другой класс evidence, поэтому он вынесен в отдельное семейство metadata_placement.

6.6. Почему combo-признаки не являются дублированием

Combo-признаки не заменяют первичные findings и не повторяют их механически. Их задача - показать, что несколько проблем встретились вместе и усиливают друг друга. Например, writable UNWIND_INFO отдельно может иметь benign-объяснение, writable handler отдельно тоже может быть особенностью сборки. Но когда writable metadata и writable handler встречаются вместе, это уже более сильный сценарий изменяемого EH/unwind-слоя.

Именно поэтому combo имеют умеренные добавочные веса +8 или +10. Они не должны удваивать score, но должны повышать убедительность связанной evidence-картины.

7. Почему именно 27, а не 10, 50 или 100

Почему не 10

Если сократить матрицу до 10 признаков, придется объединять разные причины в общие категории. Например, rf_oob, invalid_ranges, rf_overlaps и duplicate_begin стали бы одним признаком 'bad function table'. Для отчета это плохо: аналитик не поймет, что именно сломано и где проверять вручную.

В ВКР важна объяснимость. Поэтому признаки достаточно детализированы, чтобы каждый имел отдельный смысл и отдельный manual validation path.

Почему не 50

Если расширить матрицу до 50 признаков, значительная часть будет дублировать существующие группы. Например, каждую unwind anomaly можно вынести отдельно: bad opcode, bad operand, alignment, chain loop, handler OOB. Но тогда score начнет зависеть от дробления одной и той же поломки на множество мелких пунктов. Это создаст иллюзию точности и повысит шум.

Поэтому проект отделяет внутреннюю детализацию парсера от внешней evidence matrix. Внутри парсер может знать много причин, но наружу scoring получает укрупненный, понятный и стабильный finding.

Почему не 100

100 признаков были бы оправданы для универсального антивирусного классификатора: imports, strings, entropy, resources, signatures, overlay, packer fingerprints, behavioral API, network indicators. Но это другая задача. ВКР про x64 EH/unwind, поэтому расширение до 100 признаков разрушило бы фокус и усложнило защиту.

> **Сильная позиция для комиссии: 27 - это не максимум возможного, а минимально достаточный объяснимый набор для данной предметной области.**

8. Почему признаки делятся на семейства

Семейства evidence нужны, чтобы отличать независимые источники сигналов. Десять проблем в одной таблице не равны трем проблемам на разных слоях. Если повреждены PE layout, UNWIND_INFO и handler placement одновременно, это сильнее, чем если найдено несколько похожих нарушений только в одном месте.

Семейство	Что доказывает	Зачем комиссии это объяснять
table_integrity	Карта PE/function table противоречива или повреждена.	Показывает, что проект проверяет базовые инварианты, а не только эвристики.
unwind_integrity	UNWIND_INFO не читается или внутренне странен.	Это центральный предмет ВКР: возможность корректного unwind.
metadata_placement	Metadata или handler лежат в подозрительных секциях.	Показывает практическую reverse-engineering ценность.
unwind_reuse	Один UNWIND_INFO используется слишком часто.	Ловит pattern, который не является прямой битостью.
handler_dominance	Один handler доминирует в обработке.	Отражает централизацию EH-логики.

Confidence-модель использует семейства, потому что независимость слоев повышает доверие к выводу. Это еще один аргумент, что признаки отбирались архитектурно, а не как плоский список.

9. Карта 27 признаков с причиной включения

Эта таблица нужна для защиты: по каждому finding можно быстро объяснить, почему он находится в матрице и какую слепую зону закрывает.

N	Finding	Слой	Что проверяет	Почему включен
1	pe_section_raw_oob	PE layout	Проверяет, что секция не ссылается за конец файла.	Без этого нельзя доверять file offsets и чтению байтов.
2	pe_section_virtual_oob	PE layout	Проверяет, что секция не выходит за SizeOfImage.	Без этого ломается RVA-пространство образа.
3	pe_section_raw_overlap	PE layout	Проверяет пересечение секций в физическом файле.	Один байт файла не должен иметь две роли.
4	pe_section_virtual_overlap	PE layout	Проверяет пересечение секций в виртуальном образе.	Один RVA не должен принадлежать двум секциям.
5	pe_section_misaligned	PE layout	Проверяет выравнивание и базовую согласованность секций.	Это санитарный индикатор качества PE layout.
6	pe_exception_directory_oob	Exception directory	Проверяет, что directory exception table читается.	Это вход в .pdata, без него анализ невозможен.
7	no_runtime_functions	Function table	Проверяет отсутствие .pdata при наличии исполняемого кода.	Для x64 non-leaf кода это требует ручного подтверждения.
8	invalid_ranges	Function table	EndAddress <= BeginAddress.	Функция не может иметь пустой или обратный диапазон.
9	rf_oob	Function table	Begin/End выходят за SizeOfImage.	Function table не должна ссылаться на код вне образа.
10	unwind_rva_oob	Function table	UnwindInfoRVA отсутствует или выходит за образ.	Связь функция -> UNWIND_INFO становится недействительной.
11	rf_unsorted	Function table	Записи не отсортированы по BeginAddress.	Нарушает ожидаемую карту поиска функций.
12	rf_overlaps	Function table	Диапазоны функций пересекаются.	Один RIP получает две конкурирующие unwind-записи.
13	duplicate_begin	Function table	Несколько функций начинаются с одного RVA.	Конфликтует идентификация начала функции.
14	unwind_parse_fail	UNWIND_INFO	UNWIND_INFO невозможно разобрать.	Нет надежных данных для virtual unwind.
15	unwind_anomalies	UNWIND_INFO	Структура читается, но содержит логические странности.	Virtual unwind может быть формально возможен, но недостоверен.
16	unwind_reuse_moderate	Reuse	Один UNWIND_INFO заметно часто используется при наличии другого сигнала.	Ищет умеренную централизацию metadata без переобвинения.
17	unwind_reuse_strong	Reuse	Один UNWIND_INFO покрывает больше 30% большого образа.	Сильный pattern ручной нормализации или dispatcher-style metadata.
18	handler_dominance	Handler	Один handler доминирует над большинством handler-записей.	Показывает централизацию логики обработки исключений.
19	unwind_in_exec_section	Placement	UNWIND_INFO лежит в executable-секции.	Данные unwind не должны выглядеть как код.
20	unwind_unusual_section	Placement	UNWIND_INFO лежит вне ожидаемых data/xdata/pdata секций.	Ищет нестандартное размещение metadata.
21	unwind_writable_section	Placement	UNWIND_INFO лежит в writable-секции.	Metadata может изменяться после загрузки.
22	handler_nonexec_section	Placement	HandlerRVA указывает не в executable-секцию.	Обработчик должен быть исполняемым кодом.
23	handler_writable_section	Placement	HandlerRVA находится в writable-секции.	Логика обработчика потенциально изменяема.

N	Finding	Слой	Что проверяет	Почему включен
24	placement_exec_nonexec_handler_combo	Combo	Metadata в executable + handler в nonexec.	Показывает связанный конфликт размещения.
25	placement_integrity_combo	Combo	Placement anomaly + integrity problem.	Размещение становится убедительнее при структурной поломке.
26	reuse_dominance_combo	Combo	Unwind reuse + handler dominance.	Централизация metadata совпадает с централизацией handler.
27	writable_metadata_handler_combo	Combo	Writable metadata + writable handler.	И карта unwind, и обработчик потенциально изменяемы.

10. Как объяснять веса и пороги в контексте отбора

Веса не являются физическими константами. Их назначает разработчик evidence model. Но назначает не произвольно, а по силе нарушения, распространенности по таблице и вероятности benign-объяснения. Это нормальная практика для explainable heuristic analyzer: сначала задается модель предметной области, потом признаки ранжируются по степени разрушения этой модели.

Класс сигнала	Типичный вес	Логика
Жесткое нарушение границ/читаемости	+25	Если структура указывает за файл, за SizeOfImage или вообще не читается, анализатор теряет основу для вывода.
Сильная структурная или placement-проблема	+20	Сигнал очень важный, но может требовать контекста: virtual OOB, executable UNWIND_INFO, strong reuse.
Средний структурный сигнал	+15	Серьезная странность, но не всегда разрушает анализ полностью: raw overlap, unsorted table, handler dominance.
Умеренный или одиночный сигнал	+10	Требует внимания, но не должен сам по себе делать сильный verdict.
Контекстный сигнал	+8	Полезен для общей картины, но слабее прямой format/integrity ошибки.

Пороги также выбраны по смыслу. 1% используется для жестких ошибок таблиц: одна запись может быть артефактом, но больше 1% уже похоже на системность. 5% используется для более вариативных placement/anomaly-сигналов: у них больше легитимных исключений. 10% и 30% используются для reuse, потому что reuse сам по себе не запрещен. 80% для handler dominance означает не просто 'много', а подавляющее большинство.

> **Если спросят 'почему +25, а не +26', правильный ответ: потому что модель не претендует на статистическую микрокалибровку. Важны уровни доказательности, а не один балл. +25, +20, +15, +10 и +8 - это понятные классы силы evidence.**

11. Методологическая защита от обвинения в произвольности

- Каждый finding связан с конкретным объектом PE/unwind, а не с общим впечатлением.

- Каждый finding можно воспроизвести: взять поле, диапазон, секцию или счетчик и проверить руками.

- Каждый finding имеет место в архитектуре: PE layout, function table, UNWIND_INFO, handler, placement или combo.

- Слабые и спорные сигналы не получают максимальный вес и ослабляются confidence mitigations.

- Признаки общего malware analysis сознательно исключены, чтобы сохранить предметную чистоту.

- Детальные sub-anomalies не вынесены в десятки отдельных scoring items, чтобы не раздуть score искусственно.

Именно это делает набор защищаемым: он не обещает универсальную классификацию вредоносности, а дает объяснимую диагностику конкретного механизма Windows x64.

12. Кто устанавливает границы и почему за них нельзя выходить

Один из самых важных вопросов комиссии звучит так: 'почему вы считаете выход за границы ошибкой, кто эти границы установил?' Ответ: в проекте есть два разных типа границ. Первый тип - формальные границы PE/COFF и x64 unwind-структур. Они задаются форматом PE, Windows loader, компилятором/линкером и структурой Microsoft x64 exception handling. Второй тип - инженерные пороги эвристик, например 1%, 5%, 10%, 30%, 80%. Они задаются разработчиком модели, но документируются и объясняются как границы интерпретации, а не как требования PE-формата.

Формальные границы нельзя нарушать не потому, что проекту так захотелось, а потому что без них невозможно корректно сопоставить адреса, байты и структуры. Если RVA указывает за SizeOfImage, такого адреса нет внутри образа. Если raw range указывает за конец файла, таких байтов нет в файле. Если EndAddress меньше BeginAddress, это не диапазон функции. Если HandlerRVA указывает в non-executable секцию, это не выглядит как исполняемый обработчик.

Граница	Кем/чем задается	Почему нельзя нарушать	N
Размер файла	Самим файлом на диске: есть конкретное количество байтов.	Нельзя прочитать байты после конца файла. Если section raw range туда указывает, section table описывает несуществующие данные.	1
SizeOfImage	Optional Header PE, формируется линкером и используется загрузчиком Windows как размер виртуального образа.	RVA внутри PE должен относиться к области образа. Если секция или RUNTIME_FUNCTION выходит за SizeOfImage, адрес нельзя надежно сопоставить с загруженным модулем.	2, 9, 10
Section Table	PE section headers, формируются линкером или инструментом упаковки.	Секции задают перевод RVA -> section -> file offset/permissions. Пересечения делают перевод неоднозначным: один байт или один RVA получает несколько ролей.	3, 4, 5
Exception Directory	Data Directory в Optional Header. Для x64 указывает на таблицу RUNTIME_FUNCTION.	Если directory не читается или выходит за допустимые области, проект не может получить надежный массив .pdata.	6
Размер RF-записи	Формат Windows x64: запись состоит из BeginAddress, EndAddress, UnwindInfoAddress, всего 12 байт.	Если exception directory size не кратен 12 или записи не читаются, таблица функций не может быть корректно интерпретирована.	6
Begin/End	Структура RUNTIME_FUNCTION, генерируется компилятором/линкером.	Диапазон функции должен быть непустым и лежать внутри образа. Иначе RIP нельзя однозначно связать с функцией.	8, 9, 12, 13
Порядок .pdata	Практическое требование/ожидание для поиска записей в function table.	Поиск функции по адресу опирается на упорядоченную карту. Несортированность делает таблицу менее надежной и может указывать на ручное вмешательство.	11
UnwindInfoRVA	Поле RUNTIME_FUNCTION, указывает на UNWIND_INFO внутри образа.	Если указатель отсутствует или OOB, невозможно прочитать сведения о prolog, stack frame и handlers.	10, 14
Формат UNWIND_INFO	Структура Microsoft x64 unwind: header, CountOfCodes, массив unwind codes, optional handler/chain tail.	Parser должен знать, сколько байтов читать. Если CountOfCodes или tail выходят за доступный диапазон, структура не безопасна для анализа.	14, 15
Flags/opcodes	Набор допустимых значений x64 unwind ABI.	Неизвестные opcodes, плохие operands или несовместимые flags ломают смысл virtual unwind.	15
Права секций	Characteristics секции PE, которые загрузчик превращает в права памяти: execute/write/read.	UNWIND_INFO является metadata, а HandlerRVA должен вести к коду. Поэтому executable/writable/nonexec размещение меняет интерпретацию.	19-23
Пороги модели	Инженерная модель проекта, зафиксированная в evidence matrix.	Это не требования PE-формата, а границы между единичным артефактом и системным паттерном.	8-18, 20-21

12.1. Почему выход за границы файла является сильной проблемой

Raw-границы относятся к физическому файлу. Если секция заявляет PointerToRawData + SizeOfRawData больше фактического размера файла, она ссылается на байты, которых нет. Это не вопрос интерпретации и не вопрос стиля компоновки. Анализатор не может корректно прочитать то, чего физически нет. Поэтому pe_section_raw_oob получает высокий вес.

На защите можно сказать так: физическая граница установлена самим файлом. Файл имеет конечный размер. Если заголовок PE описывает секцию за этим размером, то PE layout противоречит реальности файла.

12.2. Почему выход за SizeOfImage является сильной проблемой

SizeOfImage - это граница виртуального образа, указанная в Optional Header. Загрузчик Windows использует ее как размер image mapping. Все RVA, которыми оперирует exception/unwind-механизм, должны иметь смысл внутри этого образа. Поэтому секция, function range или UnwindInfoRVA за SizeOfImage означает, что структура указывает на область, которая не является частью нормального загруженного модуля.

Важно: SizeOfImage задает не разработчик ВКР. Его задает PE-файл, обычно через линкер. Проект только проверяет, согласованы ли остальные поля с этой границей.

12.3. Почему пересечения секций опасны

Пересечение raw- или virtual-диапазонов опасно из-за неоднозначности. Если два section headers претендуют на один и тот же raw range, один байт файла имеет две секционные роли. Если две секции пересекаются по RVA, один виртуальный адрес может одновременно казаться кодом, данными или metadata. Для unwind-анализа это критично, потому что проект должен понимать, где находится .pdata, .xdata, handler code и какие у секции права.

Поэтому overlap - это не мелкая косметическая ошибка. Это конфликт карты. А если карта противоречит сама себе, любые дальнейшие выводы становятся менее надежными.

12.4. Почему диапазон функции должен быть непустым и внутри образа

RUNTIME_FUNCTION описывает диапазон инструкций функции: BeginAddress включительно и EndAddress не включительно. Если EndAddress <= BeginAddress, функция имеет нулевую или отрицательную длину. Такая запись не может корректно покрывать RIP. Если BeginAddress или EndAddress выходят за SizeOfImage, функция якобы находится вне модуля. Оба случая ломают смысл function table.

Эти границы задаются структурой RUNTIME_FUNCTION и образом PE. Проект не придумывает, где функция должна начинаться и заканчиваться. Он проверяет, что сама таблица функций не противоречит базовой логике диапазона.

12.5. Почему UNWIND_INFO должен помещаться в свой структурный формат

UNWIND_INFO - это не произвольный blob. У него есть header, Version, Flags, SizeOfProlog, CountOfCodes, unwind-коды, выравнивание и optional tail для handler или chained info. CountOfCodes сообщает, сколько code slots нужно читать. Flags определяют, есть ли handler или chain. Если этих байтов нет или combinations невозможны, parser не может безопасно восстановить смысл структуры.

Поэтому unwind_parse_fail и unwind_anomalies выбраны как признаки не потому, что parser 'строгий', а потому что virtual unwind зависит от корректного описания prolog и frame. Если описание повреждено, выход из кадра может быть неверным.

12.6. Почему handler должен указывать на исполняемую область

HandlerRVA - это адрес обработчика исключения. Обработчик является кодом, которому может быть передано управление. Поэтому если HandlerRVA попадает в non-executable или unknown section, это сильная несогласованность: поле говорит 'здесь обработчик', а section permissions говорят 'это не код'.

Writable handler не обязательно означает ошибку, но означает риск изменяемой логики. Поэтому handler_writable_section имеет medium, а не максимальный вес, и становится намного убедительнее в combo с writable metadata.

12.7. Почему процентные пороги установлены разработчиком и почему это нормально

Пороги 1%, 5%, 10%, 30% и 80% не установлены PE-спецификацией. Это инженерные границы evidence model. Их задача - отделять одиночный дефект от системного паттерна. Например, одна плохая запись в огромной таблице может быть артефактом, а десятки плохих записей говорят о повреждении или ручном формировании metadata.

Почему 1%? Потому что для жестких format/integrity ошибок даже небольшая массовость важна. Почему 5%? Потому что placement и anomalies более вариативны, там нужен более мягкий порог. Почему 10% и 30%? Потому что reuse сам по себе допустим, но заметная или огромная концентрация одного UNWIND_INFO уже подозрительна. Почему 80% для handler dominance? Потому что это не просто 'часто', а подавляющее большинство обработчиков.

Главное: эти пороги не скрыты. Они зафиксированы в коде и evidence matrix. Поэтому проект можно критиковать, настраивать и расширять. Это не черный ящик.

13. Готовые ответы на вопросы комиссии

Вопрос	Сильный ответ
Почему именно эти 27?	Потому что они покрывают полный путь x64 PE exception/unwind: от PE layout и exception directory до RUNTIME_FUNCTION, UNWIND_INFO, handler placement и корреляций. Это не случайная подборка, а послойная evidence matrix.
Почему не добавили imports/entropy/strings?	Это признаки общего malware analysis. Моя работа сфокусирована на exception/unwind metadata, поэтому я сознательно не смешивал предметные unwind-признаки с антивирусными индикаторами.
Почему COFF Header не finding?	COFF Header проверяется как precondition. Если Machine не AMD64 или Optional Header некорректен, анализ невозможен. Это не suspicious finding, а диагностика применимости инструмента.
Кто установил границы, за которые нельзя выходить?	Формальные границы задаются PE-файлом, PE/COFF-структурами, Windows loader и x64 unwind ABI: размер файла, SizeOfImage, section table, data directory, формат RUNTIME_FUNCTION и UNWIND_INFO. Проект только проверяет согласованность с ними.
Почему выход за границу считается проблемой?	Потому что структура начинает ссылаться на несуществующие байты, адреса вне образа или область с неправильной ролью. Тогда анализатор не может надежно сопоставить RVA с кодом, данными или metadata.
А процентные границы кто установил?	Процентные пороги - это не требования PE-формата, а инженерная calibration модели. Они нужны, чтобы отличать единичный артефакт от системного паттерна, и они явно задокументированы.
Почему не сделать больше признаков?	Больше не всегда лучше. Лишние признаки начали бы дублировать друг друга, повышать шум и размывать тему. Я оставил минимально достаточный набор, который можно объяснить и проверить.
Почему не меньше?	Меньше означало бы слепые зоны. Например, без placement я бы не увидел UNWIND_INFO в executable/writable секциях, а без combo не увидел бы усиление сигналов между слоями.
Почему веса задает разработчик?	Потому что это explainable heuristic model. Разработчик формализует экспертную модель, но делает ее явной: веса, пороги, mitigations и rationale документированы и проверяемы.
Почему это не антивирус?	Антивирус решает, опасен ли файл. Мой проект объясняет, что именно в EH/unwind metadata выглядит несогласованным или подозрительным. Это инструмент анализа, а не черный ящик verdict.
Можно ли добавить признаки позже?	Да. Матрица расширяема. Но новый признак должен пройти те же критерии: предметная релевантность, проверяемость, отдельный смысл, управляемый false positive и тестируемость.

14. Финальная формулировка для защиты

27 признаков выбраны на основании структуры x64 Windows exception/unwind-механизма. Я разложил механизм на слои: PE layout, exception directory, RUNTIME_FUNCTION table, UNWIND_INFO, handler layer, placement metadata и корреляции. Для каждого слоя выбраны только те проверки, которые напрямую влияют на корректность virtual unwind или помогают reverse engineer увидеть подозрительную компоновку. Признаки общего антивирусного анализа, такие как imports, entropy, strings, certificate или timestamp, сознательно не включены, потому что они не относятся напрямую к цели работы. Поэтому 27 - это не случайное число, а минимально достаточная и объяснимая evidence matrix для выбранной предметной области.

15. Вывод

Сильная защита этого решения строится на трех идеях. Первая: область анализа четко ограничена x64 PE exception/unwind. Вторая: каждый finding связан с конкретным техническим инвариантом или практическим pattern-сигналом. Третья: матрица не раздута общими malware-признаками и не скрывает решения внутри ML/черного ящика. Именно поэтому набор из 27 признаков можно объяснить, проверить, расширять и защищать.

Конец документа.